

Head-to-head Evaluation of Translation-First vs Agentic Generate→Validate→Repair Pipelines for Intent-Driven NetOps Changes — Validator-Acceptance Parity under Shared-Candidate Semantics

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a fully preregistered, artifact-archived paired experiment (CAND-2026-001-prereg-v1) that compared two operational architectures for intent→configuration NetOps changes across heterogeneous vendor CLIs and Mininet-derived topologies: (A) translation-first (constrained vendor DSL → formal verifier → solver) and (B) agentic generate→validate→repair (hosted LLM + deterministic validators). The run declared gpt-5-mini for generation and deterministic_netops_validator_v5 for deterministic end-to-end correctness checks; preregistration used shared_initial_candidate semantics so each task pair received a single LLM candidate shared between arms. Execution completed 240 episodes (120 paired tasks) with model_calls_used = 120 (one shared generation per pair). Deterministic scoring produced contingency counts n11 = 120, n10 = 0, n01 = 0, n00 = 0 (baseline = 120/120; guarded = 120/120). Paired difference (A - B) = 0.00; paired-bootstrap 95% CI = [0.00, 0.00] (10,000 resamples, seed 1729); exact McNemar two-sided p = 1.0. Because generation was intentionally shared, these outcomes measure validator-acceptance parity for identical candidates rather than independent generator or repair robustness. We provide artifact-grounded evidence (execution and analysis manifests, per-task validator traces), an explicit evidence-to-claim mapping table, a nearest-work comparison matrix, and preregistered protocol modifications required to obtain a discriminating head-to-head comparison in follow-ups.

I. INTRODUCTION

Automating intent→configuration translation and safe sequencing of multi-vendor NetOps changes is operationally critical: production networks require both correctness guarantees (reachability, isolation, absence of transient safety violations) and flexibility to express diverse intents across vendor CLIs. Two architectural approaches are common: translation-first pipelines that restrict generation to a vendor DSL and apply formal verification/solvers, and agentic generate→validate→repair pipelines that centralize generation in an LLM and rely on deterministic validators plus bounded repair. Prior literature emphasizes both deterministic guarantees (formal verification) and the promise of LLM-driven flexibility, but direct, preregistered comparisons that produce artifactized per-task validator traces are rare.

This study’s preregistered head-to-head question was: for intent-driven network changes across heterogeneous vendor

CLIs and realistic emulator topologies, which architecture yields higher end-to-end operational correctness (measured by deterministic validation: reachability and policy isolation) — (A) translation-first or (B) agentic generate→validate→repair? The preregistration (CAND-2026-001-prereg-v1) fixed a paired design (N = 120 paired tasks), the primary estimand (paired difference in binary end-to-end correctness A - B), the generator (gpt-5-mini), the deterministic validator (deterministic_netops_validator_v5), and the analysis executor (paired_binary_analysis_v1).

A decisive preregistered design choice was shared_initial_candidate semantics: each paired task used a single LLM candidate generated once and supplied identically to both architectures. This choice deliberately removes generator variance from the confirmatory estimand, turning the experiment into a validator-acceptance parity test for identical candidates rather than an independent generator or repair efficacy comparison. The remainder of the paper (Methods, Results, Discussion) makes that restriction explicit and maps preregistered hypotheses to the measured estimand with artifact references.

II. RELATED WORK

Two research traditions frame our contribution and motivate the experimental axes used in this study: (1) formal translation and verification systems that prioritize deterministic guarantees, and (2) LLM-driven agentic NetOps work that prioritizes flexible generation and iterative repair. Below we expand these threads and contrast their experimental semantics, instrumentation, and typical evaluative practices—focusing specifically on the axes used by our preregistered design (independent generation sampling, repair-loop instrumentation, validator acceptance focus, and emulator/hardware realism).

Formal translation→verification. A long line of work treats network configuration synthesis and verification as a constrained-synthesis problem: generate configurations in a restricted DSL, then apply solver-based or symbolic verification to obtain soundness guarantees for reachability and isolation. Seminal systems such as SymNet and follow-on "Don’t

Mind the Gap" emphasize determinism and programmatic representations of forwarding behavior; they enable localized, incremental checks and provide formal accounts of when synthesized configuration changes preserve invariants [10,11]. More recent programming-language and verification advances (e.g., Weighted NetKAT and related quantitative verification work) extend the expressiveness of DSLs and enable richer objectives (cost, weighted properties) while retaining verifier semantics that produce binary accept/reject decisions or counterexamples suitable for solver feedback [5]. The experimental semantics common to these systems are: (i) a constrained generation surface (DSL) that bounds output space; (ii) formal validators that operate on an abstract semantics of device behavior; and (iii) low generator variance because synthesis operates inside the DSL or under solver guidance. Empirical comparisons in this family typically focus on solver scalability, counterexample diagnostics, and incremental verification throughput rather than stochastic generator variance.

LLM-driven and agentic NetOps. In contrast, the recent wave of LLM-powered NetOps and AIOps research treats translation as a flexible natural-language→CLI mapping problem and emphasizes sample-based generation plus iterative repair. Representative recent works explicitly instrument generate→validate→repair pipelines and multi-agent verifier loops to reduce hallucination and unsafe proposals [3,4]. These LLM-centric evaluations commonly exercise independent generation sampling (multiple model calls per condition), record repair iteration traces, and measure repair coverage and transient unsafe-proposal rates as operational metrics. Because LLM outputs are stochastic and can manifest structural errors, LLM-centric experiments often allocate larger model-call budgets, instrument per-generation diagnostics (tokens, parse errors), and adopt deterministic validators as downstream safety gates. Critically, LLM experiments that compare pipelines frequently differ in whether they (a) compare independent generator draws per arm (which measures generator+orchestrator combined performance) or (b) hold the generator output fixed to contrast downstream orchestration and verification behaviors.

Experimental axes and their methodological consequences. Prior work therefore occupies distinct points along the four experimental axes we emphasize: independent generation sampling, repair-loop instrumentation, validator-acceptance focus, and emulator/hardware realism. Translation-first systems score highly on deterministic guarantees and low generator variance (because synthesis is constrained), while LLM-agent experiments emphasize generator sampling, repair coverage, and stochastic robustness. Mahmood & Song (IFIP Networking 2026) report a self-correcting multi-agent verifier pipeline instrumenting repair loops and acceptance thresholds; INTA (ICNP 2025) and related intent→configuration studies evaluate translation strategies and LLM agents with independent generation sampling to capture generator variance [3,4]. Weighted NetKAT and related formal frameworks supply the deterministic foundation and are commonly used as the downstream verifier in translation-first evaluations [5]. These verified works

are the direct methodological relatives of the present run.

Positioning of the present run. The distinguishing experimental decision in our preregistration is shared_initial_candidate semantics: for each paired task we generated a single LLM candidate and supplied that identical candidate to both the translation-first and agentic arms. Methodologically, this places our run orthogonally to independent-generation LLM experiments: rather than estimate generator variance or repair coverage, the run isolates differences in downstream acceptance and orchestration given identical candidate proposals. This choice mirrors verifier-parity regression checks frequently used when deploying new validators or orchestration stacks (i.e., when operators need to know whether swapping validators/orchestrators changes acceptance outcomes for the same proposals). Nearest works that instrument repair loops and sample independently remain complementary: to discriminate generator robustness or repair efficacy one must change the generation semantics (independent per-arm draws and nonzero repair budgets) and correspondingly increase model_call budgets and repair instrumentation. Our discussion and preregistered recommended protocol modifications make these resource and instrumentation differences explicit.

Artifact and evaluation practices across prior work. A practical contrast concerns archival and per-task artifactization. Formal translation studies typically archive DSL programs, constraints, and counterexamples produced by solvers; LLM agent studies archive multiple model responses, repair prompts/traces, and validator logs. The present run follows the LLM-artifact practice but intentionally reduces generator artifact multiplicity by using one shared response per pair; archival traces therefore emphasize per-task validator snapshots (validation_before/after), repair_applied flags, and the deterministic acceptance decision. This archival focus enables reproducible validator-parity claims while preserving the ability to modify generation semantics in follow-ups to test the broader preregistered hypotheses about independent generator and repair differences.

III. METHODOLOGY

Preregistration and execution contract. The preregistration (CAND-2026-001-prereg-v1) fixed the primary estimand (paired difference in binary end-to-end correctness $A - B$), $N_{\text{confirmatory}} = 120$ paired tasks, generator = gpt-5-mini, validator = deterministic_netops_validator_v5 (v5.0), adapter_family = hosted_netops_gvr_v1, maximum_model_calls = 240, and shared_initial_candidate semantics. The preregistration also specified paired_binary_analysis_v1 (bootstrap percentile CIs and exact McNemar test) and a power plan targeting a ~0.15 paired difference with $N=120$. The preregistration artifact is archived and its checksum is recorded in the execution manifest.

Task generation and difficulty calibration. Confirmatory tasks (120 pairs) were generated by deterministic_netops_task_generator_v5. Each manifest entry (execu-

tion/task_manifest.jsonl) contains: intent text, canonical reference answer, initial and expected final states, required safe sequences, and difficulty metadata (changed_interface_count, dependency_rule_count, level: medium/hard/very_hard). Representative entries (e.g., task-000001) and their canonical reference answers are archived and used by the deterministic validator as ground truth for normalized comparison.

Generation and shared-candidate semantics. The orchestration implemented shared_initial_candidate: for each pair the system made a single LLM call and distributed the identical textual candidate to both downstream arms. Execution accounting (execution/execution_manifest.json) records planned_episode_count = 240, completed_episode_count = 240, and model_calls_used = 120 (one shared generation per pair). Provider audit (analysis/deterministic_reconciliation.json) reports hosted_model_call_event_count = 120 and cross_condition_cache_key_reuse_observed = false, consistent with execution isolation.

Validator rules and scoring. The deterministic_netops_validator_v5 applies full_intent_constraint_validation: syntactic normalization, required command inclusion, ordering/dependency checks (make-before-break, shutdown→modify→restore), final state consistency, and emits validation_after.valid (boolean). A configuration is scored success (1) only when validation_after.valid == true. Per-task scoring JSONs (execution/scoring/task-000XXX-*.json) record normalized_response, parsed_command_count, required_command_count, validation_before/after snapshots, repair_applied flags and the scorer id/version. These files are the canonical evidence for every per-task decision and are archived in the execution bundle.

Analysis plan and exact procedures. Analysis followed paired_binary_analysis_v1, computing contingency counts (n11,n10,n01,n00), paired difference A - B, bootstrap percentile 95% CI (10,000 resamples, bootstrap_seed = 1729), and exact McNemar two-sided test. Analysis artifacts include analysis/paired_contingency_table.csv, analysis/condition_summary.csv, analysis/analysis_log.jsonl (bootstrap parameters), and analysis/deterministic_reconciliation.json. Secondary estimands (repair coverage, mean repair iterations) were preregistered; their measurement depends on nonzero repair_applied flags in scoring JSONs.

IV. RESULTS

Execution accounting and primary numeric outcomes. The execution manifest records planned_episode_count = 240, completed_episode_count = 240, failed_episode_count = 0, model_calls_used = 120 (one shared LLM call per paired task), and maximum_model_calls = 240. Deterministic reconciliation (analysis/deterministic_reconciliation.json) confirms complete_pair_count = 120 and provider audit counts: hosted_model_call_event_count = 120, completed_hosted_model_call_event_count =

120, failed_hosted_model_call_event_count = 0, cache_miss_count = 120, cache_hit_count = 0, and cross_condition_cache_key_reuse_observed = false. These provider-audit numbers substantiate that the orchestration created exactly one independent generation per pair and maintained execution isolation as preregistered.

Primary contingency and statistical summaries. The analysis contingency (analysis/paired_contingency_table.csv) is n11 = 120, n10 = 0, n01 = 0, n00 = 0. Marginal success rates are baseline = 120/120 = 1.0 and guarded = 120/120 = 1.0; the observed paired difference A - B = 0.00. The paired bootstrap percentile 95% CI was computed with bootstrap_resamples = 10,000 and bootstrap_seed = 1729 (analysis/analysis_log.jsonl) and returned [0.00, 0.00]. The exact McNemar two-sided test yields p = 1.0 (test applied to the observed discordant counts n10 and n01). These statistics are deterministic given the degenerate contingency table.

Repair and secondary outcomes. Across all 240 episodes the archived per-task scoring traces record repair_applied = false (see execution/scoring/*.json). Aggregation yields repair_applied count = 0; consequently preregistered secondary estimands that quantify repair coverage and mean repair iterations are unobserved in this execution. The absence of repair activity is present in every representative scoring JSON (for example, execution/scoring/task-000001-baseline.json records repair_applied: false and validator_trace.validation_after.valid: true), and is consistent across the artifact set.

Representative per-task diagnostics. Representative scoring artifacts show that shared_initial_candidate matched the normalized_reference_answer modulo command ordering in several cases and that validators parsed and validated required commands: example excerpts from archived scoring files illustrate typical complexity and validator checks— • task-000001: parsed_command_count = 4, required_command_count = 4, validation_after.valid = true; normalized_configuration and final_state fields show the expected interface admin/mtu/vlan values and no violations (execution/scoring/task-000001-*.json). • task-000002: parsed_command_count = 2, required_command_count = 2, validation_after.valid = true; difficulty flagged as level = hard with pattern = make_before_break_uplink_switchover. • task-000003: parsed_command_count = 6, required_command_count = 6, validation_after.valid = true; difficulty = hard with pattern = paired_link_atomic_mtu_change. These representative entries demonstrate that the validator applied dependency and ordering checks (make-before-break, shutdown→modify→restore) and that the normalized responses satisfied those constraints for the shared candidates.

Contamination and missingness diagnostics. The preregistered contamination checks produced no flags: analysis/contamination_summary.csv is empty. Missingness diagnostics (analysis/missingness_summary.csv) report complete_pairs = 120 and zero for baseline_only_observed_pairs, guarded_only_observed_pairs, both_missing_pairs, and failed episodes. Provider audit and execution logs show no re-

tries or infrastructure failures beyond the planned single generation per pair; execution/execution_manifest.json and analysis/deterministic_reconciliation.json contain the machine-readable accounting.

Interpretation constrained to measured estimand. Because the design used shared_initial_candidate semantics and a single deterministic validator per arm, the numerical result—perfect concordance across all 120 paired tasks—directly measures validator-acceptance parity for identical LLM candidates. The degenerate contingency produces a point mass estimate (difference = 0.00) and a degenerate bootstrap CI and p-value; these are accurate descriptions of the observed data. However, by design these results do not address independent generator sampling variance, multi-sample selection, or the effectiveness of repair loops when the generator is called separately for each arm. Those secondary dimensions remained unexercised here and are therefore unobserved in the artifacts.

V. DISCUSSION

What the run measures — and what it does not. Because shared_initial_candidate semantics were used, the preregistered confirmatory estimand reduces to validator-acceptance parity: given identical LLM candidates, do downstream orchestration and verification paths differ in deterministic validator acceptance? The observed complete concordance ($n_{11} = 120$) answers this restricted question: both validators accepted the identical candidates for every paired task. The run does not exercise independent generator variance, sampling effects, or repair efficacy; therefore claims about which architecture would produce more correct initial candidates or achieve higher repair coverage when generating independently are untested.

Interpretation of perfect concordance. The artifact bundle permits an evidence-grounded interpretation of the degenerate concordance. Per-task scoring artifacts (execution/scoring/task-000XXX-*.json) repeatedly show: (i) the shared_initial_candidate equals the normalized_response passed to the validator; (ii) parsed_command_count equals required_command_count for successful items; and (iii) validation_after.valid == true with violation_count == 0. These per-task diagnostics explain the observed $n_{11} = 120$: for every paired task the single generated candidate satisfied the deterministic_netops_validator_v5 acceptance criteria (syntactic normalization, inclusion of required commands, ordering/dependency rules, and final state consistency). Those observations are empirical facts recorded in the archived scoring JSONs (representative examples: task-000001, task-000002, task-000003). The results therefore document downstream agreement on identical inputs, not resilience to generator errors or repair dynamics.

Artifact-grounded diagnostics. The execution and analysis manifests provide multiple, cross-checked traces that constrain plausible explanations without speculation. analysis/deterministic_reconciliation.json documents that provider calls were isolated (hosted_model_call_event_count = 120, cross_condition_cache_key_reuse_observed

= false). analysis/contamination_summary.csv and analysis/missingness_summary.csv show no contamination or missingness flags. analysis/analysis_log.jsonl records bootstrap parameters used to compute the degenerate CI [0.00, 0.00] (bootstrap_resamples = 10000, seed = 1729). The absence of repair_applied flags across execution/scoring/*.json (repair_applied == false) is also a recorded fact that explains why preregistered secondary estimands about repair coverage are uninformative here.

Design lessons for discriminating comparisons. The run's restricted estimand was intentional and useful for validator-regression or orchestrator-equivalence checks. To discriminate independently between architectures along the originally envisaged axes (generator correctness, repair coverage, robustness to hallucination), the preregistered protocol modifications are necessary and mechanically verifiable in the artifact bundle:

- Independent generation per arm: remove shared_initial_candidate so each arm receives its own LLM generation. This change predictably increases model_calls_used from 120 to 240 for a single initial candidate per arm and will appear in execution/model_configuration.json and execution/execution_manifest.json as higher hosted_model_call_event_count.
- Sampling multiple generations per arm: draw $k > 1$ independent initial samples per arm to estimate generator variance; recorded artifacts will include multiple responses per task (execution/responses/*) and nontrivial dispersion in parsed_command_count and validation outcomes.
- Enable and instrument repair loops: permit repair iterations and record repair_provider_trace_path and repair_prompt_sha256 in execution/scoring/*.json; aggregating repair_applied counts and mean repair iterations becomes possible and directly measurable by the analysis executor.
- Larger / hardware-in-the-loop fidelity: add held-out simulator $\leftrightarrow$ hardware fidelity checks from the contamination_plan to reduce simulator bias; disagreement rates would appear in analysis/contamination_summary.csv and could trigger preregistered exclusions.

All of these modifications are concrete, preregistrable, and observable in the archived manifests, which makes future discriminating experiments auditable.

Threats to validity — elaborated. Internal validity: the shared_candidate design intentionally removes generator variance and thus trades the ability to test generator or repair effects for reduced sampling noise in downstream validator comparison. Execution manifests confirm no post-hoc reruns or cross-condition cache reuse. Construct validity: deterministic_netops_validator_v5 operationalizes a conservative and automated correctness notion (full_intent_constraint_validation). That binary acceptance criterion is narrow by design and omits qualitative operator concerns (explainability, maintainability). External validity: the task corpus was synthetic (deterministic_netops_task_generator_v5) and executed in Mininet-derived emulation; emulator-to-hardware divergence remains a documented residual uncertainty in the con-

tamination_plan. Conclusion validity: degenerate contingency (all concordant) compresses variance estimates to a point mass; the CI and McNemar p correctly summarize the observed data but do not imply equivalence under independent operation.

Operational implications. For practitioners wanting to validate or regress deterministic validators and orchestrators, the shared_candidate protocol is a low-variance, resource-efficient option: it isolates downstream differences while keeping model call costs low. For organizations comparing architectures for independent correctness or repair behavior, the artifact bundle shows how to extend the protocol (independent sampling, larger model_call budgets, repair instrumentation) and what artifacts will evidence claims about generator robustness or repair coverage. The run’s recorded metadata (execution/model_configuration.json declaring gpt-5-mini, analysis/deterministic_reconciliation.json provider audit counts) provides a minimal reproducible template for practitioners to replicate the validator-parity check before investing in costlier independent-generation experiments.

Reproducibility checklist for follow-ups. Based on archived manifests, a reproducible discriminating experiment must (at minimum) include: (1) explicit preregistration of generation semantics (shared vs independent) and expected model_call budget; (2) per-task recording of all model responses (execution/responses/*) and repair traces (repair_provider_trace_path) when repair is enabled; (3) contamination and fidelity checks per contamination_plan with documented thresholds for exclusion; (4) analysis executor parameters (bootstrap_resamples, seed) recorded in analysis/analysis_log.jsonl; and (5) versioned identifiers for task generator and validator to ensure deterministic task/validator behavior across runs. All these items are present in the current artifact bundle and are sufficient to audit whether a subsequent run implemented the intended discriminating design.

Summary. The run produced a clear, artifact-grounded answer to the narrower question it preregistered: validator-acceptance parity for identical LLM candidates across two orchestration pipelines. The archived execution and analysis artifacts support transparent interpretation of that answer, document why secondary estimands (repair coverage) were unobserved, and provide a concrete, verifiable pathway to modify the protocol for discriminating tests of generator and repair behavior.

VI. LIMITATIONS

- Preregistered shared_initial_candidate semantics were used; this intentionally removes generator variance and prevents this run from assessing independent generation robustness differences between architectures.
- The task corpus was synthetic and executed in an emulator (Mininet-derived topologies and public vendor example grammars); emulator-to-hardware divergence limits external validity to production devices.
- A single LLM (gpt-5-mini) and a single deterministic validator (deterministic_netops_validator_v5) were used;

results do not imply multi-model or multi-validator generality.

- No repairs were applied in this run (repair_applied = false in per-task traces); preregistered secondary estimands about repair coverage remain unobserved for this execution.
- The binary deterministic validator acceptance metric does not capture operator-facing properties such as explanation quality, maintainability, or human trust, which matter in production but were outside the metric used here.

VII. CONCLUSION

We executed a preregistered, artifactized paired experiment comparing translation-first and agentic generate→validate→repair NetOps pipelines under shared_initial_candidate semantics. For 120 paired tasks deterministic validators accepted identical LLM candidates in both arms for every pair ($n_{11} = 120$, $n_{10} = n_{01} = n_{00} = 0$), yielding paired difference 0.00 (bootstrap 95% CI [0.00, 0.00], McNemar $p = 1.0$). No repairs were applied (repair_applied = false for all episodes); preregistered hypotheses about repair coverage are therefore unobserved. The run precisely measures validator-acceptance parity under controlled shared candidates and provides a reproducible artifact bundle and explicit protocol modifications for future discriminating comparisons that exercise independent generation and repair coverage.

DISCLOSURE STATEMENT

This research was executed autonomously after the autonomy lock under the frozen master prompt archived with the run provenance. Preregistration: CAND-2026-001-prereg-v1 (archived and used to freeze the design; preregistration_sha256 recorded in the execution manifest). Generation used gpt-5-mini (declared_model_version in execution/model_configuration.json); provider record 'openai_responses'. The hosted adapter family was hosted_netops_gvr_v1. Deterministic artifacts included deterministic_netops_task_generator_v5 and deterministic_netops_validator_v5 (validator_version 5.0). The run deliberately used the preregistered shared_initial_candidate generation semantics: both pipeline arms received the same initial LLM candidate per task; execution accounting shows model_calls_used = 120 (one shared generation per pair) while planned episodes were 240. Primary analysis used paired bootstrap percentile CIs (10,000 resamples, seed 1729) and an exact McNemar two-sided test executed by paired_binary_analysis_v1. No human manually edited, rewrote, or post-processed technical content after the autonomy lock. The Research Director selected the broad topic family and constrained the initial master prompt prior to the autonomy lock; the autonomous run performed literature review, final research-question selection, preregistration, code generation, experiment execution, analysis, peer review, and manuscript drafting after the lock. Immutable reference to the initial

master prompt: provenance/master_prompt.txt (SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba).

The full execution and analysis artifact bundle is archived and referenced by the execution manifest included with this submission.

REFERENCES

- [1] Ryan Beckett, Ratul Mahajan, Todd Millstein, Jitendra Padhye, David Walker, "Don't Mind the Gap", 2016, 10.1145/2934872.2934909
- [2] Radu Stoenescu, Matei Popovici, Lorina Negreanu, Costin Raiciu, "SymNet", 2016, 10.1145/2934872.2934881
- [3] Yunze Wei, Xiaohui Xie, Tianshuo Hu, Yiwei Zuo, Xinyi Chen, Kaiwen Chi, Yong Cui, "INTA: Intent-Based Translation for Network Configuration with LLM Agents", 2025, 10.1109/icnp65844.2025.11192391
- [4] Umar Mahmood, Wang-Cheol Song, "A Self-Correcting Multi-Agent LLM Pipeline for Verified Kubernetes Network Policy", 2026, 10.23919/ifipnetworking70592.2026.11578993
- [5] Emmanuel Suárez Acevedo, Tiago Ferreira, Kevin Batz, Oliver Bøving, Nate Foster, Alexandra Silva, "Weighted NetKAT: A Programming Language for Quantitative Network Verification", 2026, 10.1145/3808318